

Task 1 :

```
import java.util.*;
```

```
import java.util.function.Function;
```

```
import java.util.stream.Collectors;
```

```
class Main {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        int n = sc.nextInt();
```

```
        List<Integer> salaries = new ArrayList<>();
```

```
        for (int i = 0; i < n; i++) {
```

```
            salaries.add(sc.nextInt());
```

```
        }
```

```
        Function<Integer, Integer> increaseSalary = salary -> salary +  
        (salary * 10 / 100);
```

```
        List<Integer> updatedSalaries = salaries.stream()
```

```
.map(increaseSalary)  
.collect(Collectors.toList());
```

```
for (int salary : updatedSalaries) {  
    System.out.print(salary + " ");  
}  
}  
}
```

Output :

```
5
1000 2000 3000 4000 5000
1100 2200 3300 4400 5500
```

Task 2

```
import java.util.*;
```

```
import java.util.stream.Collectors;
```

```
class Main {
```

```
    static class Reading {
```

```
        String sensorId;
```

```
        double temperature;
```

```
        Reading(String sensorId, double temperature) {
```

```
            this.sensorId = sensorId;
```

```
            this.temperature = temperature;
```

```
        }
```

```
    }
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        int n = sc.nextInt();
```

```
List<Reading> readings = new ArrayList<>();
```

```
for (int i = 0; i < n; i++) {
```

```
    String sensorId = sc.next();
```

```
    double temperature = sc.nextDouble();
```

```
    readings.add(new Reading(sensorId, temperature));
```

```
}
```

```
Map<String, Double> result = readings.stream()
```

```
    // 1. Filter temperature greater than 50
```

```
    .filter(r -> r.temperature > 50)
```

```
    // 2 & 3. Group by sensor ID and calculate average
```

```
    .collect(Collectors.groupingBy(
```

```
        r -> r.sensorId,
```

```
        Collectors.averagingDouble(r -> r.temperature)
```

```
    ));
```

```
    // 4. Sort by average temperature in descending order
```

```
    result.entrySet().stream()
```

```
        .sorted(Map.Entry.<String,  
Double>comparingByValue().reversed())  
        .forEach(entry ->  
            System.out.println(entry.getKey() + " " +  
entry.getValue())  
        );  
    }  
}
```

Output :

```
7
S1 60
S2 40
S3 70
S4 80
S5 50
S6 30
S7 90
S7 90.0
S4 80.0
S3 70.0
S1 60.0
```

Task 3 :

```
import java.util.*;
```

```
class Main {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        int n = sc.nextInt();
```

```
        int[] arr = new int[n];
```

```
        for (int i = 0; i < n; i++) {
```

```
            arr[i] = sc.nextInt();
```

```
        }
```

```
        int currentSum = arr[0];
```

```
        int maxSum = arr[0];
```

```
        for (int i = 1; i < n; i++) {
```

```
            currentSum = Math.max(arr[i], currentSum + arr[i]);
```

```
        maxSum = Math.max(maxSum, currentSum);  
    }  
  
    System.out.println(maxSum);  
}  
}
```

Output :

```
8
-2 -3 -4 -5 1 2 5 6
14
```

Task 4 :

```
import java.util.*;

class Main {

    // Create LPS (Longest Prefix Suffix) array
    static int[] createLPS(String pattern) {

        int[] lps = new int[pattern.length()];

        int len = 0;
        int i = 1;

        while (i < pattern.length()) {

            if (pattern.charAt(i) == pattern.charAt(len)) {
                len++;
                lps[i] = len;
                i++;
            } else {
                if (len != 0) {
                    len = lps[len - 1];
                } else {
                    lps[i] = 0;
                    i++;
                }
            }
        }
    }
}
```

```

        }
    }
}

return lps;
}

```

// KMP pattern searching

```
static void search(String text, String pattern) {
```

```
    int[] lps = createLPS(pattern);
```

```
    int i = 0;
```

```
    int j = 0;
```

```
    while (i < text.length()) {
```

```
        if (text.charAt(i) == pattern.charAt(j)) {
```

```
            i++;
```

```
            j++;
```

```
        }
```

```
        if (j == pattern.length()) {
```

```
            System.out.print((i - j) + " ");
```

```

        j = lps[j - 1];
    } else if (i < text.length()
        && text.charAt(i) != pattern.charAt(j)) {

        if (j != 0) {
            j = lps[j - 1];
        } else {
            i++;
        }
    }
}
}

```

```

public static void main(String[] args) {

```

```

    Scanner sc = new Scanner(System.in);

```

```

    String text = sc.nextLine();

```

```

    String pattern = sc.nextLine();

```

```

    search(text, pattern);

```

```

}

```

```

}

```

Output :

```
AABAACAADAABAABA
```

```
AABA
```

```
0 9 12
```

Task 5 :

```
import java.util.*;

class BankAccount {

    int balance = 0;

    void deposit(int amount) {

        balance = balance + amount;

    }

    void withdraw(int amount) {

        balance = balance - amount;

    }


    int getBalance() {

        return balance;

    }

}


class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        sc.nextLine();
```

```
BankAccount account = new BankAccount();

for (int i = 0; i < n; i++) {

    String operation = sc.nextLine();

    String[] parts = operation.split(" ");

    String type = parts[0];
    int amount = Integer.parseInt(parts[1]);

    if (type.equals("Deposit")) {
        account.deposit(amount);
    }
    else if (type.equals("Withdraw")) {
        account.withdraw(amount);
    }
}

System.out.println(account.getBalance());
}
}
```

Output :

```
3
Deposit 5000
Withdraw 2000
Deposit 1000
4000
```

Task 6 :

```
import java.util.*;
```

```
class Driver {
```

```
    String name;
```

```
    Driver(String name) {
```

```
        this.name = name;
```

```
    }
```

```
}
```

```
class Rider {
```

```
    String name;
```

```
    Rider(String name) {
```

```
        this.name = name;
```

```
    }
```

```
}
```

```
class Vehicle {
```

```
    String number;
```

```
    Vehicle(String number) {
```

```
        this.number = number;
    }
}
```

// Parent class

```
abstract class Trip {
    double distance;

    Trip(double distance) {
        this.distance = distance;
    }

    abstract double calculateFare();
}
```

// Bike ride

```
class Bike extends Trip {

    Bike(double distance) {
        super(distance);
    }

    double calculateFare() {
```

```
        return distance * 5;
    }
}
```

// Auto ride

```
class Auto extends Trip {
```

```
    Auto(double distance) {
        super(distance);
    }
```

```
    double calculateFare() {
        return distance * 12;
    }
}
```

// Cab ride

```
class Cab extends Trip {
```

```
    Cab(double distance) {
        super(distance);
    }
```

```
double calculateFare() {  
    return distance * 12;  
}  
}
```

```
class Main {  
    public static void main(String[] args) {  
  
        Scanner sc = new Scanner(System.in);  
  
        int n = sc.nextInt();  
  
        for (int i = 0; i < n; i++) {  
  
            String type = sc.next();  
            double distance = sc.nextDouble();  
  
            try {  
  
                if (distance <= 0) {  
                    throw new IllegalArgumentException("Invalid distance");  
                }  
  
            }  
        }  
    }  
}
```

```
Trip trip;
```

```
if (type.equals("Bike")) {
```

```
    trip = new Bike(distance);
```

```
}
```

```
else if (type.equals("Auto")) {
```

```
    trip = new Auto(distance);
```

```
}
```

```
else if (type.equals("Cab")) {
```

```
    trip = new Cab(distance);
```

```
}
```

```
else {
```

```
    throw new IllegalArgumentException("Invalid ride type");
```

```
}
```

```
System.out.println((int) trip.calculateFare());
```

```
} catch (IllegalArgumentException e) {
```

```
    System.out.println("Invalid booking");
```

```
}
```

```
}
```

```
}
```

```
}
```

Output :

```
3
Bike 10
Cab 15
Auto 8
50
180
96
```

Task 7 :

```
import java.util.*;

class Main {

    public static void main(String[] args)

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        Map<String, Integer> frequency = new LinkedHashMap<>();

        for (int i = 0; i < n; i++) {

            String hashtag = sc.next();

            frequency.put(hashtag,
                frequency.getDefault(hashtag, 0) + 1);
        }

        for (Map.Entry<String, Integer> entry : frequency.entrySet()) {
            System.out.println(entry.getKey() + " " + entry.getValue());
        }
    }
}
```

Output :

```
5
java
python
java
ai
python
java 2
python 2
ai 1
```

Task 8 :

```
import java.util.*;
```

```
class Main {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        int n = sc.nextInt();
```

```
        int m = sc.nextInt();
```

```
        // Create graph
```

```
        List<List<Integer>> graph = new ArrayList<>();
```

```
        for (int i = 0; i <= n; i++) {
```

```
            graph.add(new ArrayList<>());
```

```
        }
```

```
        // Read roads
```

```
        for (int i = 0; i < m; i++) {
```

```
            int u = sc.nextInt();
```

```
            int v = sc.nextInt();
```

```
// Undirected graph
graph.get(u).add(v);
graph.get(v).add(u);
}

int source = sc.nextInt();
int destination = sc.nextInt();

// BFS
boolean[] visited = new boolean[n + 1];
Queue<Integer> queue = new LinkedList<>();

queue.add(source);
visited[source] = true;

while (!queue.isEmpty()) {

    int current = queue.poll();

    if (current == destination) {
        System.out.println("YES");
        return;
    }
}
```

```
}

for (int next : graph.get(current)) {

    if (!visited[next]) {
        visited[next] = true;
        queue.add(next);
    }
}

}

System.out.println("NO");

}

}
```

Output :

```
5 4
1 2
2 3
3 4
4 5
1 5
YES
```

Task 9 :

```
import java.util.*;
```

```
class Calculator {
```

```
    int add(int a, int b) {
```

```
        return a + b;
```

```
    }
```

```
    int divide(int a, int b) {
```

```
        if (b == 0) {
```

```
            throw new ArithmeticException("Cannot divide by zero");
```

```
        }
```

```
        return a / b;
```

```
    }
```

```
}
```

```
class Main {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
int a = sc.nextInt();

String operator = sc.next();

int b = sc.nextInt();


Calculator calculator = new Calculator();


try {

    int result;

    if (operator.equals("+")) {

        result = calculator.add(a, b);

        // Unit test

        assert result == 30;

    } else if (operator.equals("/")) {

        result = calculator.divide(a, b);

        // Unit test

        assert result == a / b;

    } else {
```

```
        System.out.println("Invalid operator");  
        return;  
    }
```

```
        System.out.println("Test Passed");
```

```
    } catch (AssertionError e) {  
        System.out.println("Test Failed");
```

```
    } catch (ArithmeticException e) {  
        System.out.println("Test Failed");  
    }
```

```
}
```

```
}
```

Output :

Test Passed

Task 10 :

```
import java.util.*;
```

```
class LoginSystem {
```

```
    // Valid login details
```

```
    boolean login(String username, String password) {
```

```
        // Boundary value validation
```

```
        if (username.length() < 3 || username.length() > 20) {
```

```
            return false;
```

```
        }
```

```
        if (password.length() < 6 || password.length() > 20) {
```

```
            return false;
```

```
        }
```

```
        // Check username and password
```

```
        return username.equals("admin") &&
```

```
            password.equals("admin123");
```

```
    }
```

```
}
```

```
class Main {  
    public static void main(String[] args) {  
  
        Scanner sc = new Scanner(System.in);  
  
        int n = sc.nextInt();  
  
        LoginSystem system = new LoginSystem();  
  
        for (int i = 0; i < n; i++) {  
  
            String username = sc.next();  
            String password = sc.next();  
  
            try {  
  
                boolean result = system.login(username, password);  
  
                if (result) {  
                    System.out.println("SUCCESS");  
                } else {  
                    System.out.println("FAILURE");  
                }  
            }  
        }  
    }  
}
```

```
    } catch (Exception e) {  
        System.out.println("FAILURE");  
    }  
}  
}  
}
```

Output :

```
3
admin admin123
ab 123
user pass
SUCCESS
FAILURE
FAILURE
```

UseCase 1 :

```
import java.util.Scanner;

public class VehicleEntryManagement {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int levels = 3;

        int slots = 5;

        boolean[][] occupied = new boolean[levels][slots];

        System.out.println("==== Vehicle Entry Management  
====");

        System.out.print("Enter Vehicle Number: ");

        String vehicleNo = sc.nextLine();

        int assignedLevel = -1;

        int assignedSlot = -1;

        // Find nearest available parking slot
        for (int i = 0; i < levels; i++) {
            for (int j = 0; j < slots; j++) {
                if (!occupied[i][j]) {
                    assignedLevel = i + 1;
                    assignedSlot = j + 1;
                    occupied[i][j] = true;
                    break;
                }
            }
        }
    }
}
```

```

        }
    }
    if (assignedLevel != -1) {
        break;
    }
}

// Display result
if (assignedLevel == -1) {
    System.out.println("\nParking Full! No slot available.");
} else {
    System.out.println("\n----- Vehicle Details -----");
    System.out.println("Vehicle Number : " + vehicleNo);
    System.out.println("Parking Level : " + assignedLevel);
    System.out.println("Parking Slot : " + assignedSlot);
    System.out.println("Status      : Parking Allocated
Successfully");
}
sc.close();
}
}

```

Output :

```
===== Vehicle Exit Management =====
```

```
Enter Vehicle Number: TN01AB1234
```

```
Enter Entry Time (in hours): 9
```

```
Enter Exit Time (in hours): 12.5
```

```
Vehicle removed from parking successfully.
```

```
----- Parking Details -----
```

```
Vehicle Number      : TN01AB1234
```

```
Parking Duration    : 3.5 hours
```

```
Parking Fee         : Rs. 80.0
```

Vehicle Entry Management — Output

```
===== Vehicle Entry Management =====
```

```
Enter Vehicle Number: TN01AB1234
```

```
----- Vehicle Details -----
```

```
Vehicle Number : TN01AB1234
```

```
Parking Level  : 1
```

```
Parking Slot   : 1
```

```
Status        : Parking Allocated Successfully
```



ParkingAnalytics - Java

```
===== Parking Analytics =====
Enter occupied slots in Level 1: 3
Enter occupied slots in Level 2: 5
Enter occupied slots in Level 3: 2

Enter number of vehicles for each hour
0:00 - 2
1:00 - 1
2:00 - 0
3:00 - 1
4:00 - 2
5:00 - 3
6:00 - 5
7:00 - 8
8:00 - 12
9:00 - 15
10:00 - 14
11:00 - 11
12:00 - 10
13:00 - 9
14:00 - 8
15:00 - 7
16:00 - 6
17:00 - 5
18:00 - 4
19:00 - 3
20:00 - 2
21:00 - 2
22:00 - 1
23:00 - 1

----- Parking Analytics -----
Most Occupied Level : Level 2
Peak Parking Hour   : 10:00
Total Parking Slots : 15
Occupied Slots      : 10
Available Slots     : 5

Process finished with exit code 0
```

===== Priority Parking =====

Vehicles are processed according to priority.

Parking Slot 1

Vehicle Number : TN02BB2222

Vehicle Type : Emergency

Priority : 1

Parking Slot 2

Vehicle Number : TN05EE5555

Vehicle Type : Emergency

Priority : 1

Parking Slot 3

Vehicle Number : TN01AA1111

Vehicle Type : VIP

Priority : 2

Parking Slot 4

Vehicle Number : TN04DD4444

Vehicle Type : VIP

Priority : 2

Parking Slot 5

Vehicle Number : TN03CC3333

Vehicle Type : Normal

Priority : 3

